Actividad 3: Implementación del voto electrónico e incorporación de AES

Teoría de Códigos y Criptografía

Adrián Molina Martínez
Alejandro Ortega Ramón
Gabijus Bagocius
Manuel Freniche Morales

Departamento de Informática

Universidad de Almería

Almería, 29 de octubre de 2026

Índice de Contenidos

0. Enunciado 1

Ejercicio 1: Implementación del voto electrónico 1

Ejercicio 2: Incorporación de AES 7

0. Enunciado

Actividad 3. Desarrollo de código e incorporación de AES al voto electrónico

1. Desarrollo del código que implemente el voto electrónico que se analizó en la Actividad 1.
2. Incorporación de AES en todas las comunicaciones y en el cifrado de la fila de control de la urna.

Todas las capturas y salidas de este informe proceden de una misma ejecución del ciclo completo: se prepara la elección, votan siete de los diez votantes (V01, V03, V04 y V07 votan Sí; V02 y V05, No; V06, en blanco), se intenta un doble voto, se cierra, se recuenta y V01 verifica su voto.

Ejercicio 1: Implementación del voto electrónico

Organización del código

El proyecto está dividido en cinco capas. Las dependencias van en un solo sentido (`ui` → `services` → `core`, `crypto`, `storage`), de modo que el modelo del voto no sabe nada de criptografía ni de ficheros, y la interfaz no toca nunca una clave.

Módulo	Contenido
`core/scheme.py`	Parámetros públicos y aritmética módulo P: evaluación del polinomio, fila de voto, sumas por columna, recuento y búsqueda de la fila de un votante.
`core/election.py`	Censo, urna, fases de la elección y tabla de permisos de escritura por rol. Objetos inmutables: cada cambio devuelve uno nuevo.
`crypto/aes_gcm.py`	AES-256-GCM (de la Actividad 2), ahora con datos asociados (AAD).
`crypto/channel.py`	Canal seguro: tramas AES-GCM ligadas a canal, sentido y número de secuencia.
`crypto/randomness.py`	Único origen de aleatoriedad (`secrets`): claves, nonces, α, polinomios, columnas.
`services/entities.py`	Entidades: Administrador, SKGA, Recolección, Recuento y Votante.
`services/sealing.py`	Cifrado con K_b de la fila de control y de las marcas del censo.
`services/channels.py`	Enlaces cifrados entre entidades y registro del tráfico.
`services/election.py`	Fachada que ejecuta el ciclo completo y persiste cada paso.
`storage/json_store.py`	Almacén público de urna y censo, versionado y con escritura atómica.
`ui/voting_window.py`	Ventana de la votación.

Tabla 1: Módulos del proyecto

Parámetros del esquema

Se mantienen los parámetros provisionales del proyecto: 10 votantes, referéndum de dos opciones y mesa de 4 miembros. Toda la aritmética se hace en $\mathbb{Z}_P$ con el primo de 256 bits de la curva NIST P-256:

$$P = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1.$$

Cada elemento se serializa en 32 bytes, que es justo lo que cabe en un cifrado AES de dos bloques. La urna tiene $N = 10$ columnas (una por votante) y cada votante recibe un polinomio aleatorio de grado 11, $p(x) = a_0 + a_1 x + \cdots + a_{11} x^{11}$, con $a_{11} \neq 0$. Los códigos de voto $C_{\text{Sí}}$ y C_{No} son dos elementos aleatorios distintos y no nulos de $\mathbb{Z}_P$; el voto en blanco usa el código 0.

Entidades implementadas

Cada entidad de la Actividad 1 es una clase con solo los datos que le corresponden. Las entidades se comunican exclusivamente por mensajes JSON que viajan cifrados (Ejercicio 2).

- **Administrador** (`Administrator`): crea el censo V01. . . V10, marca a V01. . . V04 como mesa M1. . . M4 y genera $C_{\text{Sí}}$ y C_{No}. Lo envía a la SKGA. También es quien cierra la votación.
- **SKGA** (`Skga`): genera la clave de la elección K_b (AES-256), un identificador de elección `eid` de 16 bytes y los 10 puntos α_m distintos y no nulos. Inicializa la urna con la fila de control $E_{K_b}(0)$ en las 10 columnas y entrega K_b a la Recolección por un canal cifrado.
- **Recolección** (`Collector`): atiende a cada votante, comprueba el censo, elige su columna secreta h_t y su polinomio, y añade el voto a la urna, la fila de control y la marca del censo en una sola operación.
- **Recuento** (`Counter`): recibe K_b solo en el momento del recuento, descifra el control, calcula el resultado y descarta la clave.
- **Votante** (`Voter`): recibe p, los α y h_t, devuelve $p(\alpha_{h_t}) + c$ y se queda con su recibo (p, h_t, c) para verificar después.
- **Urna** y **Censo**: ficheros públicos `urna.json` y `censo.json`.

La mesa electoral y la SKRA (reparto de K_b en trozos con umbral 3 de 4) no se implementan todavía: en esta entrega K_b vive solo en la memoria de la SKGA, que es quien la entrega al Recuento. Este punto se completará cuando se incorpore la compartición de secretos.

Urna y censo

La urna pública tiene la estructura diseñada en la Actividad 1:

Campo	Contenido
`eleccion`, `p`, `c_si`, `c_no`	Identificador, primo y códigos de voto (públicos).
`fase`	`abierta`, `cerrada` o `escrutada`.
`alfas`	Fila 1: los diez puntos $\alpha_1, \ldots, \alpha_{10}$.
`control`	Fila 2: $E_{K_b}(A_m)$ para $m = 1, \ldots, 10$, cada una de 60 bytes en hexadecimal.
`votos`	Una fila por voto: $p(\alpha_1), \ldots, p(\alpha_{h_t}) + c, \ldots, p(\alpha_{10})$.
`resultado`	$C[1.,10]$ y los totales, solo tras el recuento.

Tabla 2: Contenido de `urna.json`

En el censo cada votante tiene `id`, `nombre`, `mesa` y `estado`. El estado está vacío hasta que vota; entonces guarda la marca $E_{K_b}(h_t)$, de 60 bytes. Así cualquiera ve *quién* ha votado, pero no en qué columna.

Atomicidad. Cada cambio (preparar, votar, cerrar, recontar) escribe una carpeta nueva `v0001`, `v0002`, ... con la urna y el censo completos, y solo al final sustituye el puntero `actual.json` con `os.replace`, que es atómico. Si algo falla a mitad, la carpeta nueva se borra y la versión anterior sigue vigente: nunca puede quedar un voto en la urna sin la marca en el censo, ni al revés. En la ejecución de prueba se generaron 10 versiones: preparación, 7 votos, cierre y recuento.

Permisos. `core/election.py` contiene la tabla de quién puede escribir qué, tal como se fijó en la Actividad 1:

Cambio	Rol autorizado	Fase exigida
Crear el censo	Administrador	sin elección
Inicializar la urna	SKGA	sin elección
Añadir un voto y su marca	Recolección	abierta
Cerrar la votación	Administrador	abierta
Publicar el resultado	Recuento	cerrada

Tabla 3: Permisos de escritura sobre urna y censo

Al publicar el resultado la urna vuelve a calcular los totales a partir de $C[m]$: un resultado construido a mano con totales inventados se rechaza.

Matemática del voto y del recuento

Para el votante t, con polinomio p_t, columna h_t y código c_t, la fila que se añade a la urna es

$$V_t[m] = \begin{cases} p_t(\alpha_m) + c_t & \text{si } m = h_t, \\ p_t(\alpha_m) & \text{si } m \neq h_t, \end{cases} \qquad (\text{mod } P).$$

A la vez, la Recolección suma a la fila de control el polinomio *sin* el código: $A[m] \leftarrow A[m] + p_t(\alpha_m)$. En el recuento, para cada columna,

$$C[m] = \sum_t V_t[m] - A[m] = \sum_t p_t(\alpha_m) + \sum_{t\,:\,h_t=m} c_t - \sum_t p_t(\alpha_m) = \sum_{t\,:\,h_t=m} c_t .$$

Como cada columna se asigna a un único votante, $C[m]$ es el código de quien la usó, o 0 si nadie la usó. Se clasifica: $C_{\text{Sí}} \to$ Sí, $C_{\text{No}} \to$ No, $0 \to$ en blanco, otro valor $\to$ nulo. Sin $A[m]$ cada fila de voto es indistinguible de valores aleatorios; por eso la fila de control es lo único que hay que cifrar.

Código

Inicio de la votación de un votante en la Recolección (extracto de `Collector.offer`): se descifran las marcas para saber qué columnas están ocupadas, se descifra el control y se eligen h_t y p con el generador seguro.

```
used = {
    sealing.open_element(self._kb, e.mark, sealing.mark_aad(eid, e.id))
    for e in census.entries if e.mark is not None
}
n = len(urn.rows)
control = tuple(
    sealing.open_element(self._kb, c, sealing.control_aad(eid, m, n))
    for m, c in enumerate(urn.control)
)
free = scheme.free_columns(used)
h_t = free[randomness.random_below(len(free))]
coeffs = tuple(randomness.random_below(scheme.P) for _ in range(scheme.DEGREE))
     + (
    _random_nonzero(),
)
```

Recuento (`scheme.tally`):

```
columns = tuple((x - y) % P for x, y in zip(b, a))
return Result(
    columns=columns,
    yes=columns.count(c_yes),
    no=columns.count(c_no),
    abstain=columns.count(0),
    null=sum(1 for c in columns if c not in (0, c_yes, c_no)),
)
```

Ciclo completo en la aplicación

1. Preparación. Al pulsar *Preparar elección* el Administrador crea censo y códigos, la SKGA genera K_b, `eid` y los α, y se escribe la versión 1. La fila *control* (en marrón) ya

contiene diez cifrados de 0, todos distintos porque cada uno usa un nonce nuevo. En el panel *Tráfico de red* aparecen las dos primeras tramas cifradas: Administrador → SKGA y SKGA → Recolección.

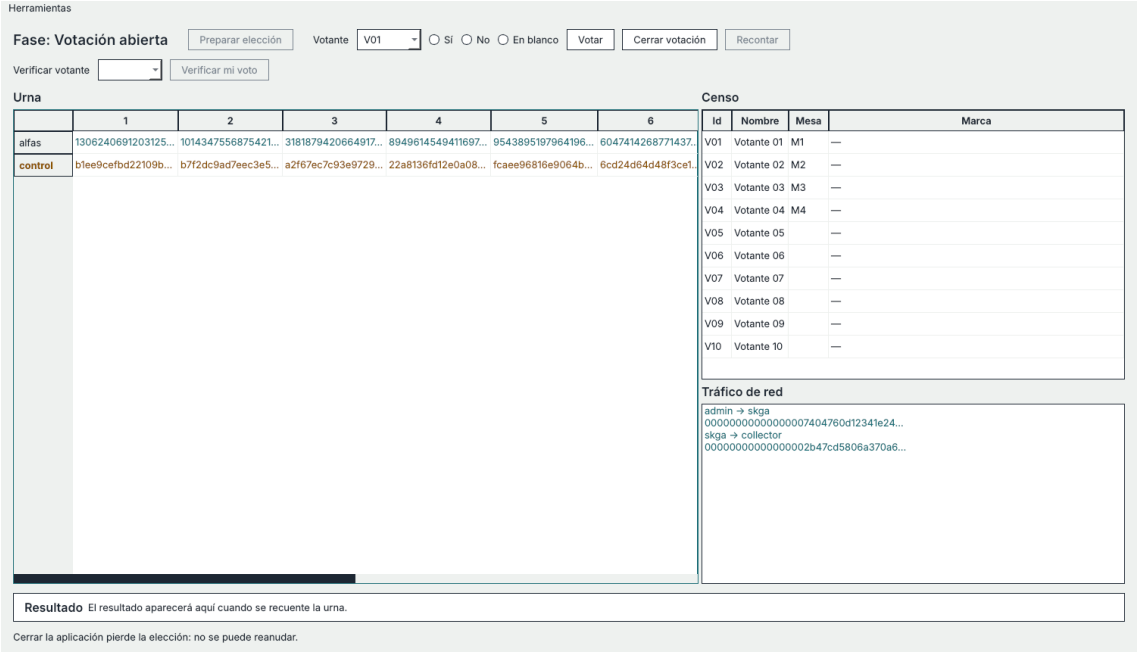


Figura 1: Elección preparada: urna con α y control cifrado, censo sin marcas

2. Votación. Se elige el votante y su opción y se pulsa *Votar*. Cada voto abre un canal nuevo entre votante y Recolección con tres mensajes: petición, oferta (p, α, h_t) y respuesta $p(\alpha_{h_t}) + c$. Tras los siete votos la urna tiene siete filas de valores que parecen aleatorios, la fila de control ha cambiado en todas las columnas y el censo muestra la marca cifrada de cada votante.

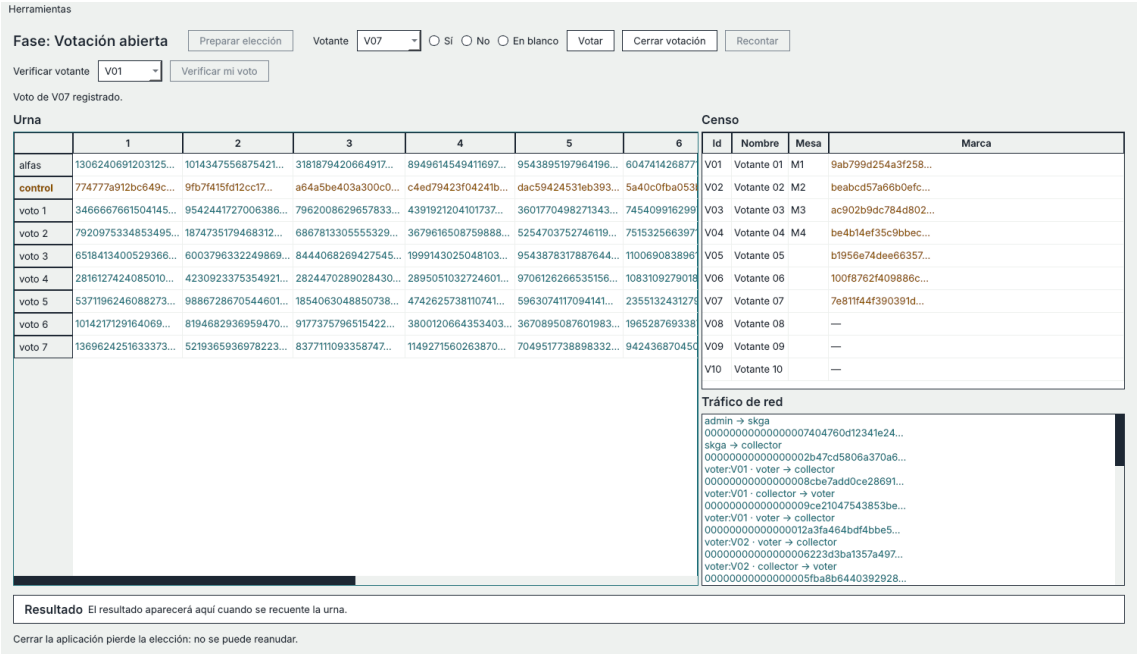


Figura 2: Urna y censo tras siete votos

3. Doble voto y votante no censado. Si V01 intenta votar otra vez, la Recolección ve su marca en el censo y rechaza la petición antes de enviarle ningún polinomio. Lo mismo ocurre con un identificador que no está en el censo (V99). El mensaje es el mismo en ambos casos, para no revelar si el votante existe:



Figura 3: Intento de doble voto de V01: «No es posible registrar el voto.»

4. Cierre. *Cerrar votación* pasa la urna a la fase cerrada; desde ese momento cualquier voto se rechaza (se probó con V08) y solo queda habilitado *Recontar*.

5. Recuento. La SKGA envía K_b al Recuento por un canal nuevo, se descifra la fila de control, se calcula $C[m]$ y se publica el resultado: **Sí 4, No 2, en blanco 4, nulos 0**. Coincide con los votos emitidos: 4 Sí, 2 No, 1 en blanco y 3 columnas sin usar (V08, V09 y V10 no votaron), que también dan $C[m] = 0$.

6. Verificación individual. Con su recibo, V01 busca en la urna pública la fila que coincide con $p(\alpha_m)$ en todas las columnas y con $p(\alpha_{h_t}) + C_{\text{Sí}}$ en la suya. No necesita K_b. La aplicación encuentra la fila 1 y comprueba que $C[h_t]$ es su código:

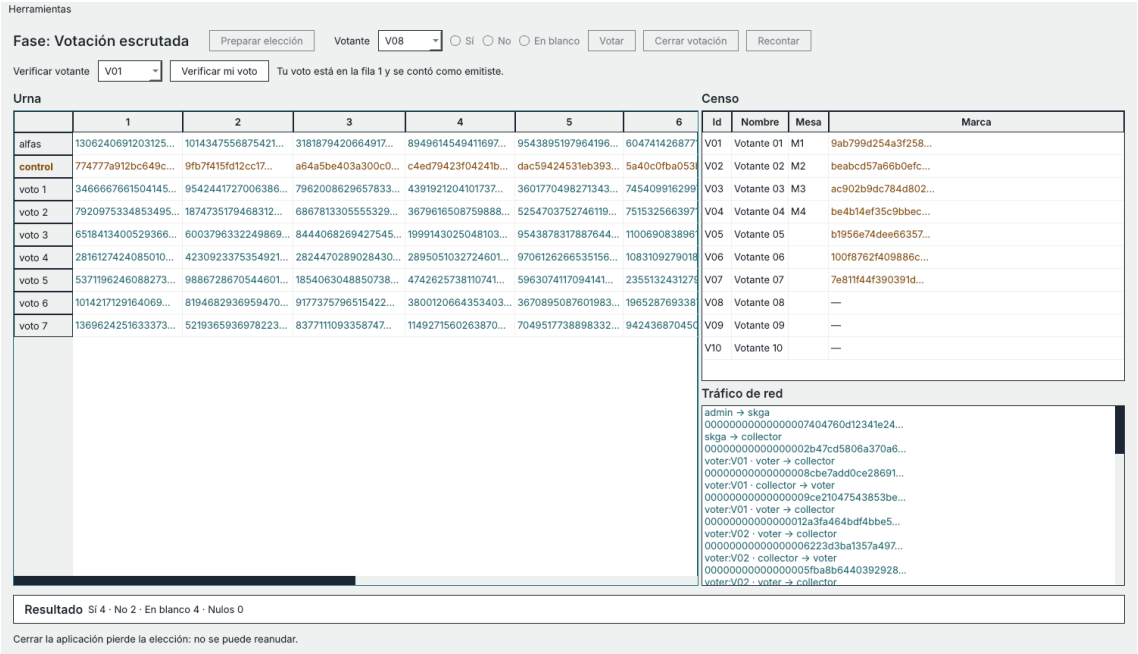


Figura 4: Resultado publicado y verificación de V01

Salida

Resultado escrito en `urna.json` (valores abreviados). Las columnas 3–6 valen 0; las 1, 2, 8 y 9 valen $C_{\text{Sí}}$ y las 7 y 10, C_{No}:

```
"resultado": {
  "C": ["49735208063086538575...", "49735208063086538575...", "0", "0", "0",
    "0",
        "10654841404326623221...", "49735208063086538575...",
        "49735208063086538575...", "10654841404326623221..."],
  "si": 4, "no": 2, "abstencion": 4, "nulos": 0
}
```

Ejercicio 2: Incorporación de AES

Primitiva: AES-256-GCM con datos asociados

Se reutiliza el módulo de la Actividad 2: AES con clave de 256 bits en modo GCM, nonce de 96 bits nuevo en cada cifrado y etiqueta de autenticación de 128 bits. El formato es

$$\text{nonce (12 B)} \parallel \text{cifrado } (n \text{ B}) \parallel \text{etiqueta (16 B)}.$$

GCM es un cifrado autenticado: además de ocultar el contenido, cualquier cambio en el dato cifrado hace que el descifrado falle sin entregar nada. Para esta actividad se añadió un parámetro *AAD* (datos asociados): datos que no se cifran ni se guardan, pero sí entran en

la etiqueta. Si al descifrar no se pasa exactamente el mismo AAD, la etiqueta no verifica. Esto permite *atar* cada cifrado a su contexto (canal, columna, votante. . .).

```
def encrypt(key: bytes, plaintext: bytes, *, aad: bytes = b"") -> bytes:
    _check_key(key)
    # Nonce generado aquí y no recibido: repetir (clave, nonce) en GCM revela
    el XOR
    # de los contenidos y permite falsificar etiquetas.
    nonce = randomness.random_bytes(NONCE_SIZE)
    return nonce + AESGCM(key).encrypt(nonce, plaintext, aad or None)
```

Todos los AAD se construyen igual: una lista de campos, cada uno precedido por su longitud en 4 bytes, empezando por la etiqueta de versión `evoting/1` y un dominio (`channel`, `control` o `marca`). El prefijo de longitud evita ambigüedades (`"ab"+"c"` y `"a"+"bc"` dan AAD distintos) y el dominio impide usar un cifrado de un tipo como si fuera de otro.

AES en todas las comunicaciones

Enlaces. Ninguna entidad habla con otra en claro. Cada conversación abre un *enlace* con una clave AES-256 nueva generada al abrirlo, y la clave se descarta al cerrarlo. Los enlaces de la elección son:

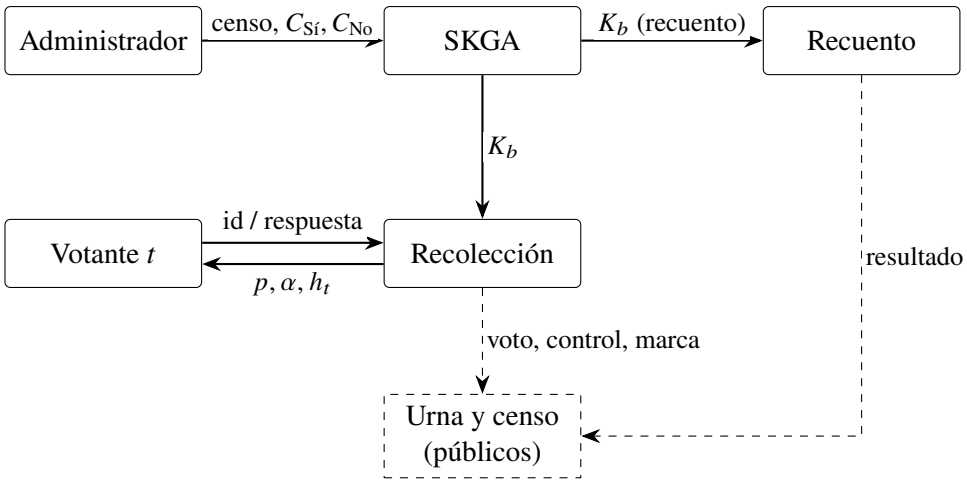


Figura 5: Enlaces cifrados con AES (líneas continuas) y escrituras en los ficheros públicos (discontinuas)

Enlace	Sentido	Contenido en claro	Trama
`admin->skga`	Admin → SKGA	censo inicial y códigos de voto	896 B
`skga->collector`	SKGA → Recolección	`{"kb": <64 hex>}`	110 B
`voter:Vxx`	Votante → Recolección	`{"id": "V01"}`	49 B
`voter:Vxx`	Recolección → Votante	12 coeficientes, 10 α, h_t	1851 B
`voter:Vxx`	Votante → Recolección	`{"respuesta": ...}`	131 B
`skga->counter`	SKGA → Recuento	`{"kb": <64 hex>}`	110 B

Tabla 4: Mensajes cifrados del ciclo y tamaño de su trama en la ejecución de prueba

La oferta al votante es la más sensible: p y h_t son los secretos que permiten reconocer su fila en la urna, y la respuesta revela su voto a quien conozca p. Por eso ninguno de los dos viaja nunca en claro.

Formato de trama. Cada mensaje JSON se cifra en el canal seguro (`crypto/channel.py`) con el formato

$$\text{secuencia (8 B)} \parallel \text{nonce (12 B)} \parallel \text{cifrado} \parallel \text{etiqueta (16 B)},$$

y el AAD de cada trama contiene el identificador del canal (nombre y 8 bytes aleatorios), el sentido (`a>b` o `b>a`) y el número de secuencia. Por ejemplo, la petición de V08 (13 bytes en claro) ocupa 8 + 12 + 13 + 16 = 49 bytes.

```
def seal(self, plaintext: bytes) -> bytes:
    if self._key is None:
        raise ChannelError("Canal cerrado.")
    seq = self._send_seq
    header = seq.to_bytes(SEQ_SIZE, "big")
    data = aes_gcm.encrypt(self._key, plaintext, aad=self._aad(self._send_dir,
    header))
    self._send_seq = seq + 1
    return header + data
```

Al recibir, primero se verifica la etiqueta y después que la secuencia sea la esperada. De este modo el canal resiste, además de la lectura, estos ataques activos:

Prueba	Qué se hace	Resultado
Alteración	Se cambia un bit del cifrado de una trama	Rechazada
Reenvío	Se entrega dos veces la misma trama válida	Rechazada la 2.ª
Reflexión	Se devuelve al emisor una trama que él mismo envió	Rechazada
Canal cerrado	Se intenta enviar tras cerrar el enlace	«El canal está cerrado.»

Tabla 5: Pruebas sobre el canal (mensaje mostrado: «Mensaje rechazado por el canal.»)

Tráfico observable. El registro de tráfico solo guarda las tramas cifradas, en hexadecimal: es lo que vería alguien que escuche la red. En la ejecución se registraron 25 tramas (2 de preparación, 21 de los siete votos, 1 del doble voto de V01 y 1 de V99), y ninguna

contiene datos legibles. Los 8 primeros bytes son el número de secuencia, que va en claro porque el receptor lo necesita, pero está autenticado por el AAD:

```
skga -> collector
0000000000000000 2b47cd5806a370a60acf1db2 ab35290ea450afe3bfa152e616c33b64...
  secuencia          nonce                 cifrado (74 B) + etiqueta (16 B)
```

Hasta K_b viaja cifrada: entre SKGA y Recolección, y entre SKGA y Recuento, con la clave del enlace correspondiente.

Cifrado de la fila de control de la urna

La fila de control guarda $A[m] = \sum_t p_t(\alpha_m)$. Es el único dato que permitiría calcular resultados parciales, así que se guarda cifrada con K_b (`services/sealing.py`). Cada celda es un elemento de $\mathbb{Z}_P$ de 32 bytes cifrado con AES-256-GCM; con nonce y etiqueta ocupa 12 + 32 + 16 = 60 bytes.

El AAD de la celda m es `(evoting/1, control, eid, m, n)`, donde n es el número de filas de voto que hay en la urna en ese momento:

```python
def control_aad(eid: str, m: int, n: int) -> bytes:
    """AAD de la celda de control: eleccion eid, columna m y n filas de voto.
    """
    return _aad(b"control", eid, str(m), str(n))

def seal_element(kb: bytes, x: int, aad: bytes) -> bytes:
    return aes_gcm.encrypt(kb, x.to_bytes(scheme.ELEMENT_SIZE, "big"), aad=aad)
```

Cada dato del AAD bloquea un ataque concreto:

- **`m` (columna):** no se pueden intercambiar celdas de control entre columnas.
- **`n` (número de votos):** no se puede volver a poner un control antiguo (*rollback*), porque fue sellado con otro n. Esto liga el control al número de filas de voto de la urna: añadir o quitar una fila hace que el control ya no abra.
- **`eid`:** no se puede copiar el control de otra elección.

Actualización con cada voto. La Recolección descifra las diez celdas con (m, n), les suma $p_t(\alpha_m)$ y las vuelve a cifrar con $(m, n+1)$ y nonces nuevos. La fila de voto, el control resellado y la marca del censo se confirman juntos en una versión nueva:

```
response = _element(reply.get("respuesta"))
row = scheme.vote_row(offer.coeffs, urn.alphas, offer.h_t, response)
shifts = tuple(scheme.evaluate(offer.coeffs, a) for a in urn.alphas)
control = scheme.add_columns(offer.control, shifts)
eid, n = urn.eid.hex(), len(urn.rows) + 1
sealed = tuple(
    sealing.seal_element(self._kb, v, sealing.control_aad(eid, m, n))
    for m, v in enumerate(control)
)
mark = sealing.seal_element(
    self._kb, offer.h_t, sealing.mark_aad(eid, offer.voter_id)
)
```

Marca del censo. Con el mismo mecanismo se cifra la marca $E_{K_b}(h_t)$ que la Actividad 1 preveía en el censo, con AAD `(evoting/1, marca, eid, id_votante)`. La Recolección la necesita para saber qué columnas están ocupadas (y retomar la votación tras un fallo), pero nadie sin K_b puede saber la columna de cada votante. Al estar ligada al identificador, la marca de V01 no se puede copiar a V02.

Salida

Pruebas de manipulación sobre la urna tras los siete votos. En todos los casos la apertura falla con el mismo mensaje fijo, «No se pudo abrir el elemento: clave incorrecta, posición distinta o dato alterado.», que no revela nada del contenido:

Prueba	Qué se hace	Resultado
Bit alterado	Se cambia un bit de la celda de control de la columna 1	No abre
Intercambio	Se intenta abrir la celda de la columna 2 como si fuera la 1	No abre
Rollback	Se usa el control de la versión 2 (1 voto) con la urna de 7 votos	No abre
Clave distinta	Se abre con una clave AES aleatoria en lugar de K_b	No abre
Marca copiada	Se abre la marca de V01 como si fuera de V02	No abre

Tabla 6: Pruebas de manipulación de la fila de control y del censo

Además se hizo una prueba de extremo a extremo en una segunda elección: tras dos votos y el cierre, se intercambiaron en el disco las celdas de control de las columnas 1 y 2 y se pidió el recuento. El Recuento abre todo antes de calcular, así que se detuvo con el error anterior, no publicó ningún resultado y la urna siguió en fase *cerrada*.

Qué protege AES en el esquema

Propiedad (Actividad 1)	**Cómo contribuye AES**
Equidad	Sin K_b no se puede descifrar el control, así que no hay resultados parciales antes del cierre.
Privacidad	p, h_t y la respuesta viajan cifrados; la marca del censo oculta h_t.
Confidencialidad e integridad de las comunicaciones	Todas las tramas van cifradas y autenticadas; se rechazan alteraciones, reenvíos y reflexiones.
Integridad de los ficheros	Cualquier cambio en el control o en las marcas impide el recuento.
No reutilización	La marca cifrada del censo identifica a quien ya votó.

Tabla 7: Propiedades del sistema reforzadas por AES

Limitaciones conocidas

- Las entidades se ejecutan en un mismo proceso y los canales son en memoria. La clave de cada enlace se genera al abrirlo y se reparte a sus dos extremos; no hay todavía un intercambio de claves real (está previsto con Kyber).
- No se autentica a las entidades ni a los votantes: los roles están fijados en el código. Las firmas (Dilithium) y los certificados quedan para más adelante.
- K_b solo existe en memoria, sin reparto entre la mesa: si se cierra la aplicación la elección no se puede reanudar (la ventana lo avisa). Se resolverá con la SKRA y la compartición de secretos.
- Las filas de voto no se cifran porque no hace falta: sin p y sin $A[m]$ son indistinguibles de valores aleatorios. Sí se puede ver cuántos votos hay en la urna.
- El voto en blanco usa el código 0, igual que una columna sin usar, por lo que en el resultado se suman ambos.